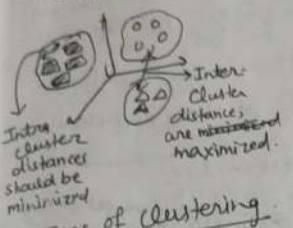
(*) Clustering / Cluster Analysis

→ Clustering is a big unsolved prob^m.
→ Objective is to detect meaningful groups insights into data.



(*) Various Distance Measures

(1) Euclidean Distance (Dissimilarity Measure)
- → L_2 Norm $= \|x-y\|_2$
- → L_1 Norm (Manhattan Distance) $= \|x-y\|_1$
- → p- Norm $= \sum_{d=1}^{D} \left(|x_d - y_d|^p\right)^{1/p}$ (p can be fraction)

(2) Cosine distance (Similarity Measure)

$$d_{cos}(x,y) = \frac{x^T y}{\|x\| \, \|y\|}$$

[$\cos\theta$ increases if similarity increases]

(*) Types of Clustering

① Partitional Algo's.

② Hierarchical " (eg. Dendogram (Tree like structure) — ↳ The similarity b/w 2 objects in a dendogram is represented as the height of lowest internal node they share.
↳ can be used for outlier too (single isolated branch)

No need to specify no. of clusters.

- Top-down (divisive): Starting with all data in a single cluster, consider every possible way to divide the cluster into two. Choose the best division & recurse.
- Bottom-Up (agglomerative): ↳ Start with each item in its own cluster & find the best pair to merge into a new cluster. Repeat until all clusters are fused together.

→ Distance b/w 2 clusters.

① Single Linkage (nearest neighbor)
↳ Take distance of 2 closest objects in diff. clusters

② Complete Linkage (furthest neighbor)
↳ Take greatest distance b/w 2 objects.

③ Group average Linkage.
↳ Take average distance b/w all pairs of objects in 2 diff. clusters.

Partitioning Clustering

↳ Each instance is placed in exactly one of the K non-overlapping clusters.
↳ Specify no. of clusters in advance.
⇒ K-Means Clustering. → It is a partitioning prob^m & It can be solved by Lloyd's heuristic.

↳ Hierarchical clustering algo^m typically have local objectives & Partitioning algo^m typically have global objectives.

↳ Cluster Membership Matrix.

$$J_{K\text{-mean}}(Z, U) = \sum_{q=1}^{K} \sum_{j=1}^{n} u_{qj} \|x_j - z_q\|_2^2$$

($J_{K\text{-mean}}$ → Objective Func^n; Z → set of Centroids)

$$U = \begin{array}{c} \\ c_1 \\ c_2 \\ c_3 \end{array} \begin{array}{c} \begin{array}{cc} x_1 & x_2 \end{array} \\ \begin{bmatrix} 1 & 0 & \cdots \\ 0 & 1 & \cdots \end{bmatrix} \end{array}$$

$\sum U$ = Cardinality of Cluster

Need to ← Minimize.
(Local minimization Heuristic)
(Lloyd's Heuristic.

$$z_q = \frac{1}{n_q} \sum x_i \quad \left(\frac{dJ}{dz} = 0\right)$$

Algo. ① Start with random cluster centroid
② Assign each x_i to one of the clusters which is nearer
③ Recompute cluster centroids $z_q = \frac{1}{n_q} \sum x_i$
④ Repeat step ② until $\|z^{t+1} - z^t\| < \epsilon$ (tolerance)

→ Dividing data points into J regions are called Vernoi regions

→ In K-mean clustering algo, objective funct^n is non-convex with multiple local minimum.

→ Soft extension of K-means = Fuzzy C-Means. → allows non-overlapping clusters

↳ $U = \begin{bmatrix} u_{11} & \cdots & u_{1c} \\ \vdots & & \vdots \\ u_{n1} & \cdots & u_{nc} \end{bmatrix}$ $0 \le u_{ij} \le 1$ ↳ $\min J_{FCM} = \sum_{j=1}^{c} \sum_{i=1}^{n} u_{ij}^{m} \| x_i - v_j \|_2^2$

s.t. $\sum_{j=1}^{c} u_{ij} = 1, \ \forall i$

m = Fuzzy fier (user-defined)

Linear Regression Analysis

$y = a + b_1 x_1 + b_2 x_2 + \dots + b_k x_k + \varepsilon$.

Baseline Estimate $= \bar{y} = \frac{\sum y}{n}$ ↳ The Best fit line = regression line.

↳ For simple Linear Reg^n = $\hat{y} = a + bx$ (a: Intercept, b: Regression coeff)

$b = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$, $a = \bar{y} - b\bar{x}$ } we get it by minimizing $(y-\hat{y})^2$

↳ $R^2 = 1 - \frac{RSE}{TSE}$ → RSE: Residual sum of errors, TSE: Total sum of errors

If $R^2 = 75\%$ = It means 75% estimat^n error is explained or eliminated by regression model. only 25% error is remaining due to other predictor or random variance/chance.

→ For simple linear regression, $r = \sqrt{R^2}$ (Pearson correlat^n coeff)

→ Cost funct^n → $J(\theta) = \frac{1}{2m} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$ & $\min_\theta J(\theta)$ using gradient descent

→ If learning rate is too small, gradient descent is ~~so~~ slow

& " " " " large. " " can overshoot the minimum

→ For linear reg. $h_\theta(x) = \theta^T x$ $\theta_0 x_0 + \theta_1 x_1 + \dots$

Improve Learning

↳ Feature Scaling. ↳ Features have same scale ↳ Gradient descent converge faster.

↳ In PCA, Regularizat^n, Gradient descent.

→ Feature Standardizat^n → Rescale Features to have zero mean & unit variance

↳ Find Z-Score = $\frac{x-\mu}{\sigma}$

↳ Feature Normalizat^n $\frac{x - x_{min}}{x_{max} - x_{min}}$

(*) Underfit = High Bias
Overfit = High Variance

$J(\theta) = RSE + \lambda \|\theta\|_2^2$

$\lambda > 0$ Regular

In Regularizat^n, we reduce $\theta_1, \theta_2, \dots$ & can address overfitting by eliminating features.

L_2-regularizat^n pulls coeff towards zero

$Var[f] = E[(f(x) - E(f(x)))^2] = E(f^2) - E[f]^2$

Covariance of 2 r.v's X & Y measures the extent to which they vary together

(*) $Cov(X,Y) = E[(X - E(X))$

(*) Gaussian dist^n are widely
↳ Central Limit Theorem
↳ Computationally convenient
↳ Mixture of Gaussians are su to approximate wide
↳ Closely related to squar

(*) $p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\{$

(*) Variance measures h
Covariance measures t

(*) Covariance becomes ma from their mean in if X & Y are indepe

(*) Correlation is a di (-1 to +1) ↳ $\frac{\sigma_{ij}}{\sigma_i \sigma_j}$

(*) Mahalanobis Distance $\|x-y\|^2_{\Sigma^{-1}}$ = (norm)

Independent R.V.
$P(X,Y) = P(X)P(Y)$
$P(X|Y) = P(X)$

(*) Conditiona

(*) Principle of Max^m Likelih

→ Suppose we know the di parameters θs. For Be
↳ Choose a value of μ th

(*) ML for Bernoulli → L(

(*) ML for Gaussian → $\hat{\theta}$ (L

$l(\theta) = \log(L(\theta))$

↳ $\hat{\mu}_{MLE} = \frac{1}{n} \sum x$ & $\hat{\sigma}$

Unbiased Estimat $E[\hat{\theta}] = \theta$

Av

(*) $Cov(X,Y) = E[(X-E(X))(Y-E(Y))]$ (*) Gaussian Dist^n $= \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$

(*) Gaussian dist^n are widely used in ML due to
- ↳ Central Limit Theorem
- ↳ Computationally convenient
- ↳ Mixture of Gaussians are sufficient to approximate wide range of dist^n
- ↳ Closely related to squared loss.

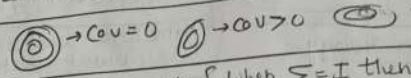
If $X \sim N_d(\mu, \Sigma)$ → d×d dimensional covariance matrix
↓ d-dimensional mean-vector.
then each dimension of x is univariate normal.
but converse is not true.

(*) $p(x) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\{-\frac{1}{2}\underbrace{(x-\mu)^T\Sigma^{-1}(x-\mu)}_{\text{Mahalanobis distance funct}^n \text{ (quadratic form)}}\}$

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1d} \\ \sigma_{21} & \sigma_2^2 & & \\ \vdots & & \ddots & \\ & & & \sigma_d^2 \end{bmatrix}$$

(*) Variance measures how much X varies around the expected value.
Covariance measures the strength of linear relationship b/w X & Y.
(*) Covariance becomes more +ve for each pair of values which differ from their mean in the same direct^n. & similar for −ve values.
if X & Y are independent then $Cov(X,Y) = 0$.

(*) Correlation is a dimensionless measure of linear dependence.
(−1 to +1) ↳ $\frac{\sigma_{ij}}{\sigma_i \sigma_j}$



(*) Mahalanobis Distance $\|x-y\|^2_{\Sigma^{-1}} = (x-y)^T\Sigma^{-1}(x-y)$ (norm) {When $\Sigma = I$ then it becomes Euclidean distance}
↳ Ellipse consists of points that are equi-distant to centre w.r.t mahalanobis distance.

(*) Independent R.V.
$P(X,Y) = P(X)P(Y)$
$P(X|Y) = P(X)$

(*) Conditionally Independent $P(X,Y|Z) = P(X|Z)P(Y|Z)$.

(*) Principle of Max^m Likelihood (ML)
→ Suppose we know the dist^n → $P(x;\theta)$ or $P(y|x;\theta)$, we want to determine parameters θs. For Bernoulli dist^n ⇒ $P(X=1;\mu) = \mu$ & $P(x=0;\mu) = P(0) = 1-\mu$
↳ Choose a value of μ that maximizes the likelihood given the observed data.

(*) ML for Bernoulli → $L(\mu) = \prod_{i=1}^{n} p(x_i;\mu) = \prod_{i=1}^{n} \mu^{x_i}(1-\mu)^{1-x_i}$ ⇒ $l(\mu) = \log L(\mu)$ → maximize it. ↓ $\hat{\mu}_{ML} = \frac{1}{n}\Sigma x$

(*) ML for Gaussian → $\hat{\theta}_{MLE} = \underset{\theta}{\operatorname{argmax}} \underbrace{\left(\frac{1}{2\sigma^2}\right)^n e^{-\sum_{i=1}^{n}(x_i-\mu)^2/2\sigma^2}}_{J(\theta)}$ (L(θ))

$l(\theta) = \log(L(\theta))$

↳ $\hat{\mu}_{MLE} = \frac{1}{n}\Sigma x$ & $\hat{\sigma}^2_{MLE} = \frac{1}{n}\Sigma(x_i-\hat{\mu})^2$ (Biased) $\hat{\sigma}^2_{unbiased} = \frac{1}{n-1}\Sigma(x_i-\mu)^2$

Unbiased Estimat $E[\hat{\theta}] = \theta$ | Average is a way to approximate expectat^n | $Var(ax+b) = a^2 Var(x)$

Logistic Regression (classifier)

$\sigma(z) = \frac{1}{1+e^{-z}}$ → sigmoid funct^n

$\hat{y}^{(i)} = \sigma(\theta^T x^{(i)}) = \frac{1}{1+e^{-\theta^T x^{(i)}}}$ → In $\frac{\partial C(\theta)}{\partial \theta_J}$, we get $\sigma'(\underbrace{\theta^T x^{(i)}}_{z})$

for large z, $\sigma'(z)$ can become very small & we get very slow weight update using GD (Gradient Descent)

$C(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(\sigma(\theta^T x^{(i)}) - y^{(i)})^2$

Loss Funct^n ↳ $\frac{\partial C(\theta)}{\partial \theta_J} = (\sigma(\theta^T x^{(i)}) - y^{(i)})\, x_J$

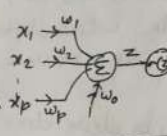
$= \frac{\partial C(\theta)}{\partial(\sigma(z))} \times \frac{\partial \sigma(z)}{\partial z} \times \frac{\partial z}{\partial \theta_J}$ (Chain Rule)

↳ $\sigma(z)(1-\sigma(z))$ ← to resolve it

$=$

So, $\frac{\partial C(\theta)}{\partial \sigma(z)} = \frac{-y}{\sigma(z)} + \frac{(1-y)}{1-\sigma(z)} \Rightarrow C(\theta) = \underbrace{-[y\log(\sigma(z)) + (1-y)\log(1-\sigma(z))]}_{\text{Binary Cross Entropy loss.}}$

→ We can build simple NN from stacked logistic regression classifiers (Neural N/w)



Naive Bayes Classificat^n

$P(C|A) = \frac{P(A|C)\,P(C)}{P(A)}$ (Posterior) → Likelihood, Prior; $P(A)$ → ignore (Same for all classes)

$P(C_J|d) = \frac{P(d|C_J)\,P(C_J)}{P(d)}$

→ Assumpt^n → Features are independent.

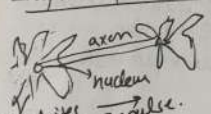
$P(d|C_J) = P(d_1|C_J)\,P(d_2|C_J)$ ---

↳ Naive Bayes is not sensitive to irrelevant features.

* Laplace smoothing: $P(A_i|C) = \frac{N_{ic}+1}{N_c+C}$

Introduct^n to Neural N/ws.

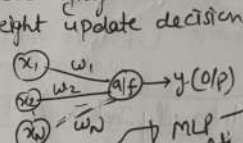
Inspired from Biology.



↳ Activat^n funct^{n}s are highly non-linear.

↳ After each weight update decision boundary gets changed.

↳ Forward Pass



(*) Performance usually tops out at 2-3 layers deep N/w except CNN

(*)

MLP (Multi layer Perceptron) → 4+3 = 7 units (Perceptrons / Neurons).

Learnable Parameters # $= (3\times4 + 4\times3) + (4+3)$ (weights) (bias)

(*) Loss funct^n defines what is mean to be close to the true sol^n.

(*) Squared (L_2) Loss $= (\hat{y}-y)^2$

L_1 - loss $= |\hat{y}-y|$

Zero-one loss $= 1[\hat{y}=y]$

Hinge loss $= \max(0, 1-y\hat{y})$ (In SVM)

In Perceptron $w = w - \nabla w$

$= w + (y_n - \hat{y})\, x_i$

Proof $L = \frac{1}{2}(y-\hat{y})^2$ with $\hat{y} = wx$ → So, $\frac{dL}{dw} = \frac{d}{dw}\{\frac{1}{2}(y-\hat{y})^2\}$

$= -(y-\hat{y})^2 \frac{dw}{dw}$

$= -(y-\hat{y})x$

$= \nabla w$

→ We use chain rule in Backpropagatn

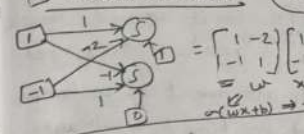
ex. $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial f_3}\frac{\partial f_3}{\partial a_3}\frac{\partial a_3}{\partial f_2}\frac{\partial f_2}{\partial a_2}\frac{\partial a_2}{w_2}$

$a = \sum wx + b$, f = activat^n fun

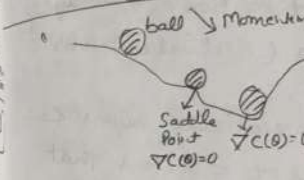
(*) Prob^m with NN → vanishing gradient → overfitting

In XOR prob^m, single decision boundary can't separate data → So, we have to use 2 layers.

Matrix Operation



$= \begin{bmatrix} 1 & -2 \\ 1 & -1 \end{bmatrix}$ [...] $= w$ ... $\sigma(wx+b)$ →



Learning Rate

↳ Reduce the learning

↳ At the beginning, we

& after several epo

Eg. $\frac{1}{t}$ decay $\Rightarrow \eta$

⇒ RProp ⇒ w^{t+1} ←

⇒ RMSProp. (Root Mean square.) w^{t+1} ←

⇒ Adagrad w^{t+1} ←

↳ If w has some

↳ " " " la

↳ RMSProp is sim

↳ $w^{t+1} \leftarrow w^t$ here.

Matrix Operation

$$= \begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$= w \quad x \quad + b$

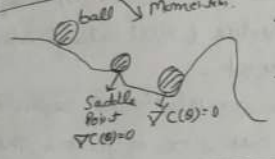
or $(wx+b) \Rightarrow \ldots$

Softmax $\rightarrow$ $z_1 \rightarrow e^{z_1}$, $z_2 \rightarrow e^{z_2}$ (for multiclass)

$y_1 = \frac{e^{z_1}}{\sum e^{z_i}}$, $y_2 = \frac{e^{z_2}}{\sum e^{z_i}}$

Probability

(*) SGD is a special case of mini-batch gradient descent with batch size 1. (generally sampling w/o replacement)



→ Movement = Negative of gradient + Momentum.

→ Momentum $v^0 = 0 \rightarrow v^1 = \lambda v^0 - \mu \nabla C(\theta^0)$

↳ learning rate.

$v^2 = \lambda v^1 - \mu \nabla C(\theta^1)$ . . .

→ Nesterov Momentum Update

$v = \mu v - \eta * dx$ → learning rate.

$x \neq = v$

Learning Rate.

↳ Reduce the learning rate by some factor every few epochs.

↳ At the beginning, we are far from a minimum, so use larger learning rate.

& after several epochs, we are close to a minimum, so we reduce " ".

Eg. $\frac{1}{t}$ decay $\Rightarrow \eta^t = \frac{\eta}{t+1}$ (*) Large gradient $\Rightarrow$ smaller learning rate.

$\Rightarrow$ RProp $\Rightarrow w^{t+1} \leftarrow w^t - \frac{\eta}{|g^t|} g^t$ → Partial derivative.

$\Rightarrow$ RMSProp. (→ Root mean square.) $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma^t} g^t$ where. $\sigma^t = \sqrt{\alpha (\sigma^{t-1})^2 + (1-\alpha)(g^t)^2}$

$\Rightarrow$ Adagrad $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma} g^t$, where σ = Average gradient of parameter w.

↳ If w has small avg gradient $\sigma \Rightarrow$ Larger learning rate

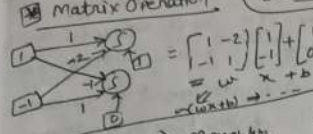
↳ " " " large " " . $\Rightarrow$ smaller " "

↳ RMSProp is similar to Adagrad.

(*) Adadelta is also a way.

Here ↳ $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma^t} g^t$, $\sigma^t = \sqrt{\frac{1}{t+1} \sum_{i=0}^{t} (g^i)^2}$

Matrix Operation



Softmax

$z_1 \rightarrow e^{z_1}$, $z_2 \rightarrow e^{z_2}$ (for multiclass)

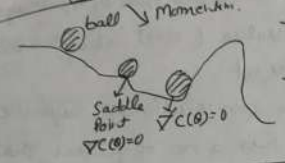
$y_1 = \frac{e^{z_1}}{\sum e^{z_i}}$, $y_2 = \frac{e^{z_2}}{\sum e^{z_i}}$ Probability

(*) SGD is a special case of mini-batch gradient descent with batch size 1. (generally sampling w/o replacement)

→ Movement = Negative of gradient + Momentum.

→ Momentum $v^0 = 0 \rightarrow v^1 = \lambda v^0 - \mu \nabla C(\theta^0)$
↳ learning rate.

$v^2 = \lambda v^1 - \mu \nabla C(\theta^1)$ - - -



→ Nesterov Momentum Update
$v = \mu v - \eta * dx$ → learning rate.
$x \mathrel{+}= v$

Learning Rate

↳ Reduce the learning rate by some factor every few epochs.

↳ At the beginning, we are far from a minimum, so use larger learning rate.

& after several epochs, we are close to a minimum, so we reduce " ".

Eg. $\frac{1}{t}$ decay $\Rightarrow \eta^t = \frac{\eta}{t+1}$. (*) Large gradient ⇒ Smaller learning rate.

⇒ RProp ⇒ $w^{t+1} \leftarrow w^t - \frac{\eta}{|g^t|} g^t$ → Partial derivative.

⇒ RMSProp. (→ Root Mean square.) $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma^t} g^t$ where, $\sigma^t = \sqrt{\alpha(\sigma^{t-1})^2 + (1-\alpha)(g^t)^2}$

⇒ Adagrad $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma} g^t$, where σ = Average gradient of parameter w.

↳ If w has small avg gradient σ ⇒ Larger learning rate

↳ " " " large " " ⇒ smaller " "

↳ RMSProp is similar to Adagrad.

(*) Adadelta is also a way.

Here ↳ $w^{t+1} \leftarrow w^t - \frac{\eta}{\sigma^t} g^t$, $\sigma^t = \sqrt{\frac{1}{t+1} \sum_{i=0}^{t} (g^i)^2}$

(*) Single layer NN can't solve XOR prob^m, 2-layer NN can solve XOR prob^m but can't solve CNN, for it, we need 3-layer NN.

(*) CNN is used in image processing, signal processing etc.

(*) CNN over MLP (Multilayer Perceptron) ⇒ Improvement of performance accuracy & some degree of invariance to distortion in I/p images.

↳ If image is shifted then feature value (pixel values) will be changed & prediction can be different.

(*) A CNN is a neural n/w with some convolutional layers & some other layers. A convolutional layer has a no. of filters that does convolutional operation.

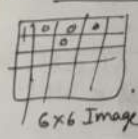
↳ A filter (set of kernels) can be used for beak detection. [Scale & shift invariance]

↳ A convolutional layer is a small odd size (generally) filter which is dragged over entire image & at every position (pixel) component-by-component product happen & response is recorded.

↳ $f * g(x) = \int_{\alpha=-\infty}^{\infty} f(\alpha)\, g(x-\alpha)\, d\alpha = \sum_{\alpha=0}^{N-1} f(\alpha)\, g(x-\alpha)$ [1D feature map]

$f * g(x,y) = \int_{\alpha=-\infty}^{\infty} \int_{\beta=-\infty}^{\infty} f(\alpha,\beta)\, g(x-\alpha, y-\beta)\, d\alpha\, d\beta$ [2D feature map]

(*) Convolution



6x6 Image

Mask → Convolutional Mask Coeff. [These are N/w parameters & to be learned]

3x3 filter

→ It detects high level & abstract features.

→ Stride means shift.

If stride=2 then shift 2 units

→ After convolution of image with filter (mask), we get feature map. & it contains high level info^n.

→ 6x6 image & 3x3 filter with stride=1 & no padding gives 4x4 feature map & if there are 2 filters, we get 2x4x4 feature map.

→ With zero padding, we get 6x6 feature map (Shape Inv.)

↳ If image has 3 channels (R, G, B), we have to use each filter with 3 kernels.

(*) Whole CNN

Image of Cat → Convol^n → Max Pooling → Convolution → Max pooling → Flattened (vector) → Fully connected Feed Forward N/w → Softmax → Cat, dog ...

Why Pooling? → Subsampling pixels will not change the object, so inference will not change. [Fewer Parameters to characterize image]

→ Maxpooling is used for undersampling & it removes redundant info^n.

↳ Divide 4x4 feature map into 2x2 blocks & keep max^m element & remove rest of the elements

(*) ReLU(z) = max(0, z)

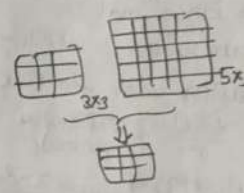
Prob^m with ReLU is, zero & these neurons ... ReLU is a special case ...

(*) Dropout

↳ It is a heuristic techn...

↳ At training time, rand... less parameters. At t... is p %, so all the we... deleted neurons at ...

(*) In Deep Learning, C... addition. The filter is ... are the same thing. ... structure of a "layer" ... structure of a "fi... The term "filter" ... stacked together.



Here, the input l... The filter is a 3x3 ... are applied to 3 c... are performed wh...

↳ Each of the per... form one channel

↳ Finally the there's a b... term. Each o/p filter ha... The bias gets adde...

(*) $ReLU(z) = max(0, z)$ ⇒ Fast to compute, Resistant to Vanishing Gradient Prob.

Prob^m with ReLU is, if z is negative then gradient becomes zero & these neurons will be eliminated.

ReLU is a special case of maxout (learnable activat^n funct^n).

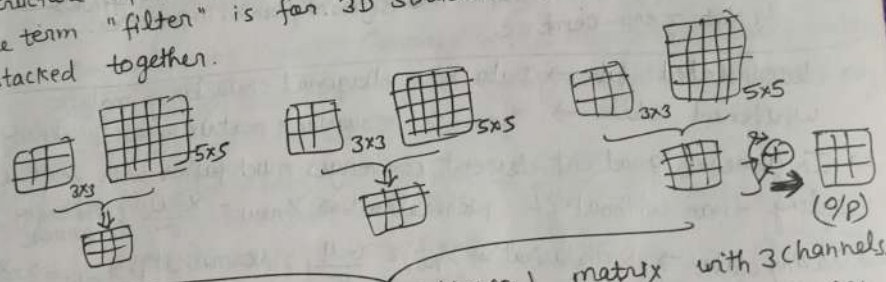
↳ It takes the max^m of a group.

(*) Dropout

↳ It is a heuristic technique to save NN from overfitting (high variance) prob^m.

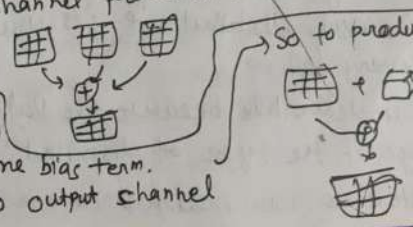
↳ At training time, randomly delete p% of neurons, so it has less parameters. At testing, no dropout & if dropout rate at training is p %, so all the weights rescaled by (1-p)% to compensate the deleted neurons at training time.. [Dropout is a kind of ensemble]

(*) In Deep Learning, Convolution is the element-wise multiplication & addition. The filter is sliding through the input channels & feature map are the same thing. "Channels" is usually used to describe the structure of a "layer." Similarly, "Kernel" is used to describe the structure of a "filter". A "kernel" refers to a 2D array of weights. The term "filter" is for 3D structure of a multiple kernels stacked together.



Here, the input layer is a 5x5x3 (tensor) matrix with 3 channels. The filter is a 3x3x3 matrix. First, each of the kernels in the filter are applied to 3 channels in the input layer separately, 3 convolutions are performed which result in 3 channels with size 3x3.

↳ Each of the per-channel processed versions are then summed together to form one channel.

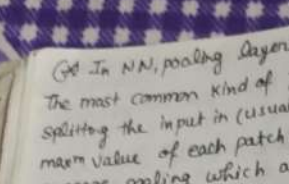


↳ So to produce final output channel

↳ Finally the there's a bias term. Each o/p filter has one bias term. The bias gets added to output channel

(*) In NN, pooling layers provide invariance to small translat^n of i/p. The most common kind of pooling is max pooling which consists in splitting the input in (usually non-overlapping) patches & outputting the max^m value of each patch. Other kinds of pooling exist. eg. mean or average pooling which all share the same idea of aggregating the input locally by applying a non-linearity to the content of some patches.

(*) Activat^n Funct^ns



⇒ Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ ✗ → Now a days, people prefer it.

⇒ $\tanh(x) =$ ✗

⇒ ReLU = $\max(0, x)$ ✓ → used to save from dead neuron prob

⇒ Leaky ReLU = $\max(0.1x, x)$

⇒ Maxout = $\max(w_1^T x + b_1, w_2^T x + b_2)$

⇒ ELU (Exponential linear unit) = $\begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$

⇒ Parametric ReLU (PReLU) = $\max(\alpha x, x)$

⇒ Sigmoid → Squashes numbers in [0,1] ⇒ Probl^s. ↳ Saturated neurons "Kill" the gradients. (also with tanh)

⇒ tanh → " " " [-1,1] ↳ Not zero centre, e^{-x} is expensive to compute

↳ zero centered. → Vanishing gradient Prob^m

⇒ ReLU → Does not saturate. (in +ve region) → computationally efficient

↳ Converges much faster than sigmoid / tanh. in practice.

↳ Not zero-centric

⇒ decorrelated data ⇒ Data has diagonal covariance matrix.

whitened data ⇒ " " covariance matrix which is identity matrix

⇒ In general, gradient descent converges much faster with feature scaling than without it. Normalizat^n ⇒ $x_{new} = \frac{x-\mu}{\sigma}$ } whitening process

⇒ Scaling data ⇒ Standarizat^n ⇒ $x_{new} = \frac{x-\mu}{\sigma}$, Normalizat^n ⇒ $x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$

(Whitening in NN)

⇒ Batch Normalizat^n (BN) ~~solves~~ reduces the prob^m of Covariate Shift. It performs whitening to the intermediate layers of the N/w.

⇒ BN reduces covariate shift. That is change in distribut^n of activat^n of a component. By using BN, each neuron's activation becomes (more or less) a Gaussian distribut^n i.e. it's usually not active, sometim a bit active, rarely very active.

⇒ Covariate shift is undesirable because the later layers have to keep adapting to the change of the type of distribut^n (instead of just to new dist^n parameter eg. new mean & variance for gaussian dist^n).

⇒ BN reduces effects

Input to every layer

w/o BN, low activat

activations in the nex

next layer & so on.

⇒ Normally, large l

parameters, which th

& lead to the model

layer is unaffected b

the parameter growth

↳ we introduce, for

$\gamma^{(k)}, \beta^{(k)}$ which

$y^{(k)} = \gamma$

scaling & shifting parameters.

BN ⇒ F

sequene

(*) GANs (General

Generative Deep Inferenc

to those in a limite

(but unknown) prob. d

→ GAN has 2 NN < G, D

→ Generator's training

& Discriminator's

→ Discriminator Training

Given a set of +ve exam

generate a set of -ve

a discriminator funct^n

⇒ Generator N/w : T

Discriminator N/w : t

→ Train jointly in minim

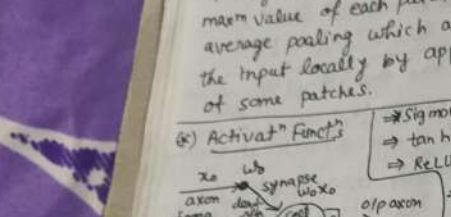
discriminate +ve & -ve data

→ Use cross-entropy

→ Use KL Divergen

(*) In NN, pooling layers provide invariance to small translat^{n}s of I/p. The most common kind of pooling is max pooling which consists in splitting the input in (usually non-overlapping) patches & outputting the max^m value of each patch. Other kinds of pooling exist. eg. mean or average pooling which all share the same idea of aggregating the input locally by applying a non-linearity to the content of some patches.

(*) Activat^n Funct^{n}s



⇒ Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ ✗ → Now a days, people prefer it.

⇒ $\tanh(x) =$ ✗

⇒ ReLU $= \max(0, x)$ ✗ → used to save from dead neuron prob.

⇒ Leaky ReLU $= \max(0.1x, x)$

✗

⇒ Maxout $= \max(w_1^T x + b_1, w_2^T x + b_2)$

⇒ ELU (Exponential linear unit) $= \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$ ✗

⇒ Parametric ReLU (PReLU) $= \max(\alpha x, x)$

⇒ Sigmoid → Squashes numbers in $[0,1]$ ⇒ Probl^s. ↳ Saturated neurons "Kill" the gradients. (also with tanh)

⇒ tanh → " " " $[-1,1]$ ↳ Not zero centre, e^{-x} is expensive to compute

↳ zero centered. → Vanishing gradient Prob^m

⇒ ReLU → Does not saturate. (in +ve region) → computationally efficient

↳ Converges much faster than sigmoid / tanh. in practice.

↳ Not zero-centric

⇒ decorrelated data ⇒ Data has diagonal covariance matrix.

whitened data ⇒ " " covariance matrix which is identity matrix

⇒ In general, gradient descent converges much faster with feature scaling than without it. Normalizat^n ⇒ $x_{new} = \frac{x-\mu}{\sigma}$ } whitening process

⇒ Scaling data ⇒ Standarizat^n ⇒ $x_{new} = \frac{x-\mu}{\sigma}$, Normalizat^n ⇒ $x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$

(Whitening in NN)

⇒ Batch Normalizat^n (BN) ~~solves~~ reduces the prob^m of Covariate Shift. It performs whitening to the intermediate layers of the N/w.

⇒ BN reduces covariate shift. That is change in distribut^n of activat^n of a component. By using BN, each neuron's activation becomes (more or less) a Gaussian distribut^n i.e. it's usually not active, sometimes a bit active, rarely very active.

⇒ Covariate shift is undesirable because the later layers have to keep adapting to the change of the type of distribut^n (instead of just to new dist^n parameter. eg. new mean & variance for gaussian dist^n).

⇒ BN reduces effects

Input to every layer

w/o BN, low activat

activations in the nex

next layer & so on.

⇒ Normally, large l

parameters, which th

& lead to the model

layer is unaffected b

the parameter growth

↳ we introduce, for

$\gamma^{(k)}, \beta^{(k)}$ which

$y^{(k)} = \gamma$

scaling & shifting parameters.

BN ⇒ F

sequene

(*) GANs (General

Generative Deep Inferenc

to those in a limite

(but unknown) prob. d

Ex: → GAN has 2 NN < G, D

→ Generator's training

& Discriminator's

→ Discriminator Training

Given a set of +ve exam

generate a set of −ve

a discriminator funct^n

⇒ Generator N/w : T

Discriminator N/w : t

→ Train jointly in minim

discriminate +ve & −ve data

→ Use cross-entropy

→ Use KL Divergen